

PROJECT WALKTHROUGH

# PathFinder

Grounded career-path recommendations and adaptive learning roadmaps for tech students

# A deep walkthrough, section by section

01

## Problem Understanding

Why career choice fails tech students today

03

## System Architecture

Three services, one contract — plus data model

05

## Key Features & Workflow

What the student experiences, end to end

07

## Challenges & Status

What was hard, what's shipped, what's next

02

## Solution Approach

A deterministic core, personalized by AI

04

## AI / ML Techniques

The scoring formula, worked examples, and guardrails

06

## Security & Privacy

Defense in depth, built in from day one

# Tech students pick careers with too little signal

## 1 Too many entry paths, no compass

Frontend, backend, data, cloud — students can't tell which track fits their interests and current skills.

## 2 Shallow, unverifiable quizzes

Generic career quizzes give a label with no breakdown, and often imply job-eligibility probabilities they can't back up.

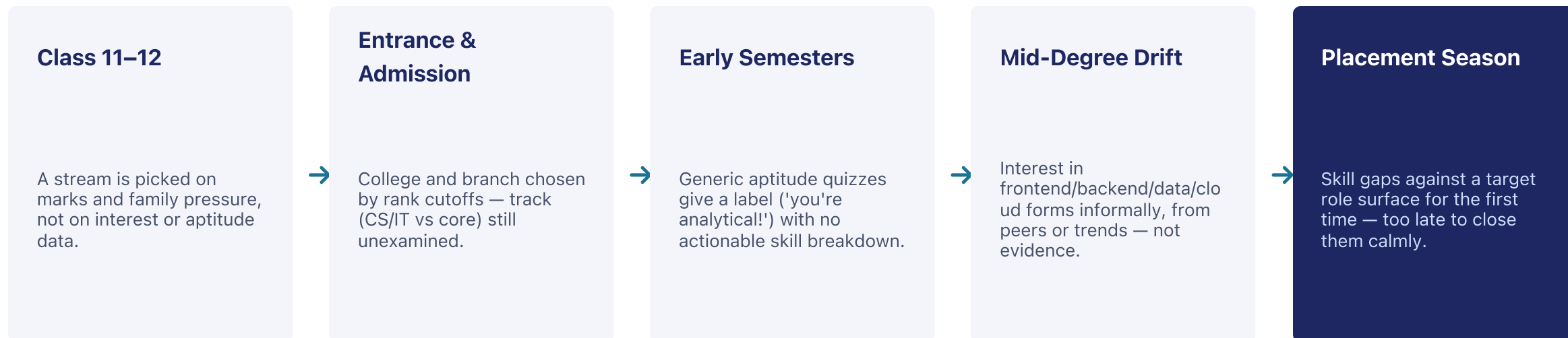
## 3 Invisible skill gaps

Students discover missing core skills only at interview stage, after months on the wrong track.

## 4 Untrustworthy AI advice

Free-form LLM guidance can invent certifications, resources, or salary claims that mislead a first-time job seeker.

## Where the information gap actually opens up



*PathFinder is built to sit as early as possible in this timeline — replacing guesswork with a transparent, revisitable score.*

# The cost of getting the first track wrong

## Cost to the student

- Months spent building the wrong skill stack before anyone notices
- Confidence hit at interview stage, right when it matters most
- Re-learning under time pressure instead of a paced plan

## Cost to existing tools

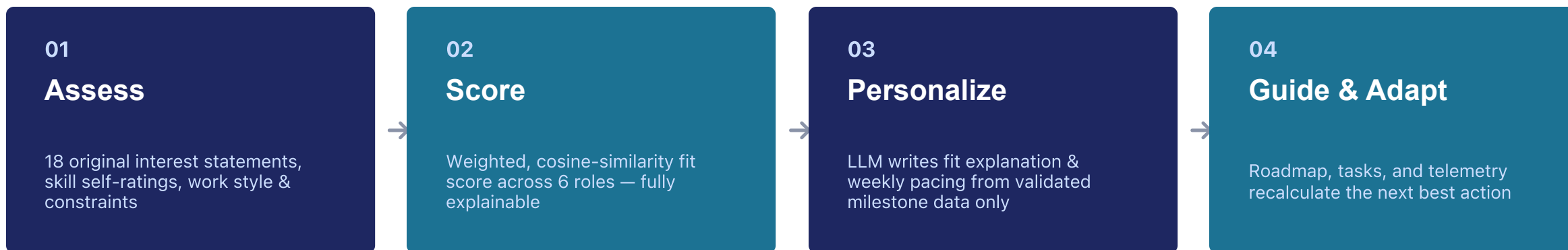
- One-shot quizzes never revisit the student as skills change
- Free-form AI chat has no memory of what was actually verified
- Neither tool shows its work — a label isn't an explanation

## What a better tool needs

- A score the student can question line by line
- A plan that updates as real skills change, not a one-time verdict
- AI that personalizes tone and pacing, never invents facts

## A deterministic core, personalized by AI — never the reverse

PathFinder scores every student against six curated roles using a transparent, hand-tuned formula. An LLM only personalizes explanations and pacing after the score is already fixed, and a template fallback keeps the product fully usable if the AI call fails.



**Guiding rule:** manual assessment is always authoritative; enrichment and AI text can inform the experience but never silently change a score or a fact.

# Three guardrails that shape every design decision

## Deterministic first, AI second

1

Every fact the student sees — the score, the breakdown, confirmed and missing skills — comes from hand-tuned formulas the team can audit line by line. The LLM is only invited in after that fact is already fixed.

## AI drafts inform, they never overwrite

2

AI-drafted assessments can propose answers, but nothing changes a recommendation until the student explicitly reviews and approves it. No background process silently edits a profile.

## The product must never look broken

3

If the LLM times out, rate-limits, or returns something that fails schema validation, a deterministic fallback takes over — same shape, same UI treatment. There is no visible 'AI error' state.

# Three deployable services, one contract between them

## Frontend · React + Vite + TypeScript + Tailwind CSS

Landing · Assessment · Results · Career Dashboard · Progress — deployed on Vercel

↓ *Supabase-authenticated REST calls (JSON)*

## Backend API · FastAPI + Pydantic

catalog · match · profile · roadmaps · tasks · intake + Q&A — JWT verified on every data call, deployed on Railway

↓ *Verified user ID, static catalog JSON, structured LLM prompt*

### Supabase

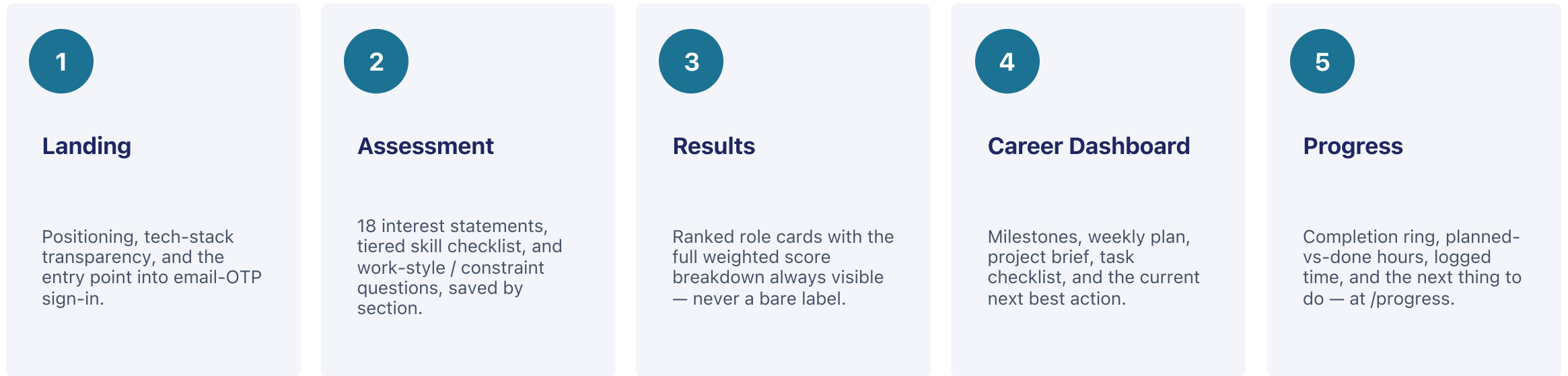
Postgres (profiles, recommendations, roadmaps, tasks) with Row-Level Security · Auth (email OTP, JWKS)

### LLM (OpenRouter)

Structured JSON for fit explanation & weekly pacing · called only after deterministic scoring · openrouter/free auto-router, key server-side only



## Five surfaces, one authenticated data path



*Stack: React + Vite + TypeScript + Tailwind CSS · Deployed on Vercel · All data calls carry a Supabase-issued JWT*

## Six FastAPI modules behind one JWT gate

### catalog

Serves versioned role and skill definitions (roles.v1.json, assessment.v1.json) — read-only, no DB round trip.

### match

Runs the deterministic scoring engine and returns the full three-term breakdown per role.

### profile

Owns assessment vectors, skill confidence, work-style, hours/week, timeline, and selected role.

### roadmaps

Generates the weekly-plan JSON — LLM-personalized when available, deterministic fallback otherwise.

### tasks

Tracks per-milestone task completion state and timestamps that drive the next-best-action logic.

### intake + Q&A

Drafts the assessment from a plain-language goal and answers grounded questions.

*Every data route verifies a Supabase JWT before touching a module. Deployed on Railway.*

# Data model: five per-user tables, one static catalog

|                   |                                                                                                 |
|-------------------|-------------------------------------------------------------------------------------------------|
| profiles          | Assessment vectors, skill confidence, work preferences, hours/week, goal text, selected role    |
| recommendations   | Persisted match cache: score + breakdown stamped to the profile version, served by GET /match   |
| roadmaps          | Weekly-plan JSON, generation mode (llm or fallback), generated timestamp                        |
| tasks             | Roadmap ID, milestone ID, task ID, completion state, completed timestamp                        |
| enrichment_events | User ID, source type, success/failure state, timestamp — never résumé text or raw API responses |

*Role, skill, milestone, and assessment definitions live in versioned backend JSON (roles.v1.json, assessment.v1.json) — not the database — so curated career content ships and reviews like code.*

# Why content is code and data is minimal

## Content ships like code

- roles, assessment, and courses ship as versioned JSON, not DB rows
- Changes go through the same review and diff process as application code
- A rollback is a redeploy, not a data migration

## Every table is access-controlled

- Row-Level Security policies sit on profiles, recommendations, roadmaps, tasks
- Enforced in Postgres itself — independent of any application-layer bug
- A user ID always comes from the verified JWT, never the request body

## Only what's needed persists

- Only approved skills and telemetry persist — never source documents
- Roadmap rows record generation mode (llm / fallback) for auditability
- No table stores raw LLM prompts or provider responses

# Matching engine: an explainable, weighted similarity model

**PathFinder Fit Score = 0.55 × Interest Similarity + 0.35 × Skill Readiness + 0.10 × Work-Style Similarity**

## Interest alignment — 55%

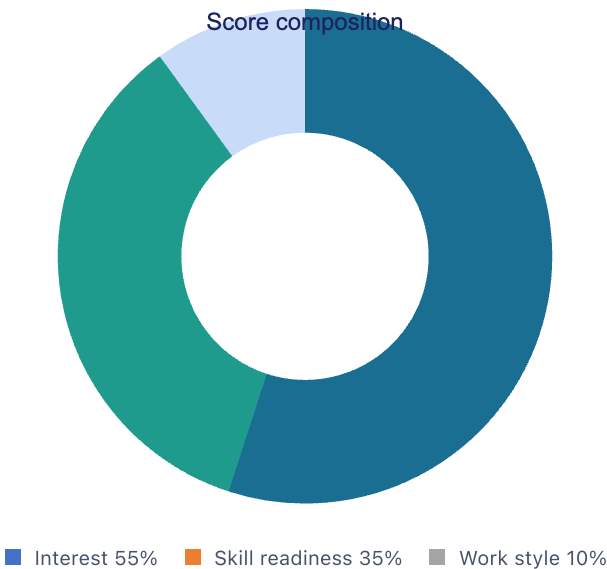
Cosine similarity between the student's 6-dimension RIASEC vector (from 18 original statements) and each role's stored vector. Magnitude-invariant, so response intensity doesn't skew the match.

## Skill readiness — 35%

Confidence per skill (none 0 → aware 0.3 → practised 0.7 → project-ready 1.0), weighted by tier (core 1.0, supporting 0.5, optional 0.25), normalized by each role's maximum possible weighted score.

## Work-style alignment — 10%

Cosine similarity on 5 dimensions: analytical, creative, collaborative, structured, systems-oriented.



*Output shown to the student: total score, the three-term breakdown, confirmed skills, and the exact missing core skills — never a claim of job-placement probability.*

# Turning three numbers into an honest recommendation

*Illustrative example — not real student data*

| Component                  | Similarity | Weighting | Contribution |
|----------------------------|------------|-----------|--------------|
| Interest similarity        | 0.82       | × 0.55    | 0.451        |
| Skill readiness            | 0.61       | × 0.35    | 0.214        |
| Work-style similarity      | 0.70       | × 0.10    | 0.070        |
| Total PathFinder Fit Score |            |           | 0.735        |

## What the student actually sees

- Backend Engineering — 73.5% fit, ranked #1 of 6
- Interest: strong (0.82) · Work style: strong (0.70)
- Skill readiness: moderate (0.61) — the limiting factor
- Confirmed core skills: REST APIs, SQL basics, Git
- Missing core skills: authentication & authorization, testing fundamentals

*The score breakdown and the missing-skill list are the same two things the LLM is later allowed to explain in plain language — it cannot change either.*

# Adaptation & guarded LLM personalization

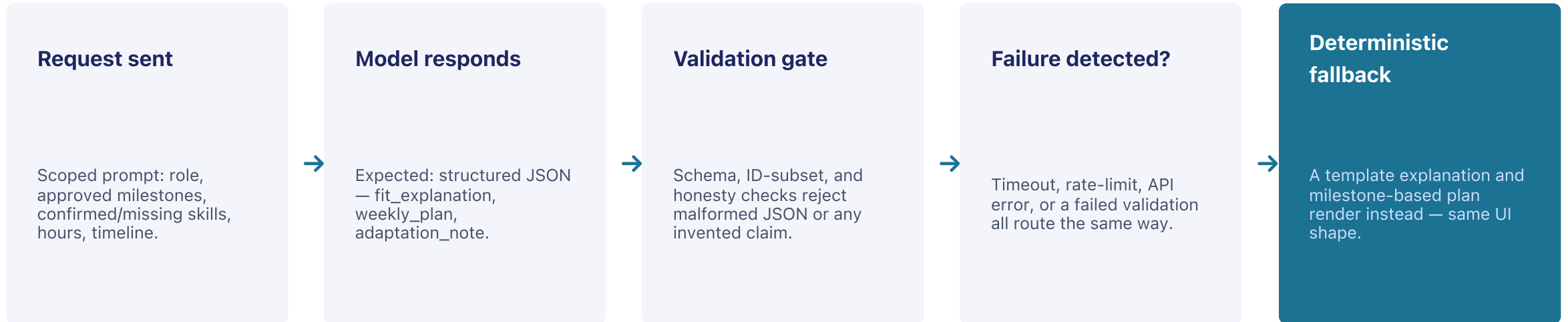
## Adaptation (rule-based)

- Per-skill state: perceived vs. demonstrated level, resulting gap and risk tier (low / medium / high)
- Career readiness % derived from weekly hours, timeline, and completed milestones
- Estimated weeks to close the gap, flagged ahead / compressed against the student's target timeline
- Deterministic "next best action" — the skill with the largest demonstrated gap, with its reason
- Telemetry feedback: task time & quiz scores promote demonstrated skills into the profile

## LLM personalization (guarded)

- Prompt scope is fixed: only the selected role, its approved milestones, confirmed/missing skills, hours, and timeline — never open-ended
- Model must return structured JSON: `fit_explanation`, `weekly_plan`, `adaptation_note`
- Pydantic schema + ID checks + honesty detectors reject invented skills, resources, roles, or timelines
- On API, validation, timeout, or rate-limit failure, a deterministic template explanation and milestone-based plan take over silently
- The dashboard is designed to never surface an AI error to the student

## What happens when the model doesn't cooperate



*The UI treats a fallback response identically to a live one — the student never sees a broken AI state, only a plan.*



## Six shipped features carrying the product forward

1

### Interest & Skill Assessment

18 RIASEC statements, tiered skill checklist, work-style and constraint questions.

2

### Transparent Fit Scoring

Ranked cards for all six roles with the full weighted breakdown, always shown.

3

### Career Path Dashboard

Five milestones, weekly plan, project brief, checklist, and next best action.

4

### Telemetry & /progress Page

Time & quiz telemetry, skill promotion, and a dedicated progress tracking page.

5

### Conversational Goal Intake

Describe your goal in plain words; the AI drafts an editable assessment you approve.

6

### Secure, Isolated Accounts

Supabase JWT auth plus Row-Level Security so one user can never read another's data.

# Student workflow: sign-in to adaptive next action



## Profile & evidence page (deferred with enrichment cut)

Deferred with the enrichment cut: GitHub URL and résumé upload. Editing the assessment anytime ships today — retake it and every cached result recomputes against the new profile.

## What we cut, and the principle that survived

1

### The cut line

We dropped résumé/GitHub enrichment to protect the core loop.

2

### The core loop

Assessment → match → roadmap → fallback had to be bulletproof before anything else.

3

### What shipped instead

Conversational intake applies the same principle: the AI proposes a draft; the student reviews every field.

4

### What remains

The unused enrichment\_events table stays in the schema — shown, not hidden.

5

### The lesson

Cutting on time is a feature: every shipped surface is tested, fallback-safe, and demo-ready.

## Defense in depth, built in from day one

### Verified identity everywhere

Every data route verifies a Supabase JWT (JWKS RS256/ES256, HS256 fallback); the user ID always comes from the token, never the request body.

### Row-Level Security

Postgres RLS policies on profiles, recommendations, roadmaps, and tasks — a defense-in-depth layer independent of application code.

### Minimal data retention (shipped)

No raw LLM prompts or provider responses are stored; goal text is length-capped and treated as untrusted data, never instructions.

### Fail-open, not fail-broken

If any LLM call fails, a deterministic fallback continues the flow — AI never blocks the core product.

### Locked-down CORS

Only the configured production and local frontend origins may call the API.

### No silent AI errors

LLM failures fall back to deterministic templates; the student never sees a broken AI state.

## Five independent layers, none of which trust the others

### Network / CORS

Only whitelisted frontend origins can call the API at all.

### Authentication

Supabase JWT verified on every data route (JWKS primary, HS256 fallback).

### Authorization

Postgres Row-Level Security enforces per-user data isolation independently of app code.

### Data minimization (shipped)

No raw LLM prompts or provider responses are stored anywhere.

### Application resilience

Fallback-safe AI and graceful degradation mean a failure never blocks or breaks the product.

*A request would need to defeat all five layers, independently, to reach another student's data or a broken experience.*

## What made this harder than it looks

### Keeping the score fair across roles

Raw weighted sums silently favored roles with longer skill lists. Fixed by normalizing skill readiness against each role's own maximum possible weighted score.

### Never letting the AI break the demo

LLM calls can time out, rate-limit, or return malformed or invented content. Solved with strict Pydantic schema validation plus a deterministic fallback that the UI treats identically to a live response.

### Verifying Supabase JWTs robustly

Key rotation and algorithm changes needed a dual path: JWKS-based RS256/ES256 as primary, HS256 shared-secret as a transition fallback.

### Enrichment vs. privacy

Résumé and GitHub enrichment had to add value without ever persisting raw source data or changing a score without explicit approval.

### A hard deadline & cut line

Required an explicit cut line: drop résumé/GitHub enrichment first if the assessment → match → roadmap → fallback core loop wasn't solid by the mid-point checkpoint.

### Catching the LLM when it lied

Live personalization is real: every AI response is checked against the profile. A weak-fit test caught invented skills and a false timeline — now permanent honesty detectors guard every call.

# What's shipped today, and what's next

## Built & tested

- Six-role catalog & assessment APIs serving versioned JSON
- Deterministic matching engine, regression-tested with one representative profile per role
- Profile, roadmap, task persistence + /progress page with fallback plan generation
- Conversational intake and grounded Q&A endpoints live
- Supabase JWT verification (JWKS + HS256 fallback) on every data route

## Next up

- Match cache with version stamps: GET /match serves the persisted result, 404 when stale
- Résumé/GitHub enrichment — deliberately cut to protect the core loop
- Row-Level Security policies verified directly against Postgres, not just the app layer
- Deploy configs in-repo: Vercel (frontend), Railway (backend), six migrations applied
- 107 backend + 5 frontend tests; lint & build clean

## Beyond the current cut line: a three-horizon vision

### Near term

- Done: live OpenRouter personalization is wired in, fallback verified with the key disabled
- Done: RLS policies verified directly against Postgres, not just the app layer
- Revisit résumé/GitHub enrichment with full approval UI if time allows

### Mid term

- Deepen the six curated roles as demand data comes in
- Add mentor or peer-matching based on shared skill gaps and timelines
- Smarter pacing: richer adaptation as telemetry data accumulates

### Long term

- A companion mobile experience for daily task check-ins
- Optional, opt-in aggregate cohort analytics for institutions
- Employer-side signal (without ever exposing individual student data)

*Framed as direction, not a committed timeline — every horizon still answers to the same guiding rule: deterministic first, AI second.*



# Thank you

PathFinder — grounded scoring, adaptive roadmaps, and AI that never speaks off-script.